

AWS Compute Services

EC2 · ECS · Fargate · Lambda — a complete comparison

Control & Responsibility Spectrum

You manage everything

AWS manages everything

EC2	ECS on EC2	Fargate	Lambda
EC2 Traditional server A full virtual machine in the cloud. You get root access to a complete Linux/Windows server. Install anything, run anything — but Cost: Pay per hour (instance on = billing on) Cold start: ~60 sec (boot)	ECS on EC2 Containers on your servers Docker containers orchestrated by ECS, running on EC2 instances you still manage. Containers give you isolation and portability; the Cost: Pay for EC2 instances (running or idle) Cold start: ~30 sec (container pull)	Fargate Serverless containers Run Docker containers with zero server management. Define your container image, CPU, and memory. AWS provisions invisible Cost: Pay per vCPU/memory-second (only when running) Cold start: ~5-10 sec (task spin-up)	Lambda Fully serverless Write a function, define a trigger, deploy. AWS runs it on demand. No containers, no servers, no idle cost. It wakes up in milliseconds, Cost: Pay per 1ms execution + requests (can be \$0/m) Cold start: < 1 sec (but cold-start latency exists)

Feature comparison

	EC2	ECS on EC2	Fargate	Lambda
No server management	✗	✗	✓	✓
Run Docker containers	✓	✓	✓	✗
Per-second billing	✗	✗	✓	✓
Auto-scales instantly	✗	✗	✓	✓
Full OS access	✓	✓	✗	✗
Persistent long tasks	✓	✓	✓	~
Zero idle cost	✗	✗	✗	✓
Works with EKS/ECS	✓	✓	✓	✗
Custom runtime/GPU	✓	✓	~	~
Event-driven triggers	✗	✗	~	✓

✓ Yes ✗ No ~ Partial / Limited

Who manages what

EC2	ECS on EC2	Fargate	Lambda
› Hardware (virtual) › OS & patches › Runtime › Scaling › App code AWS manages 0+ layers below	› EC2 fleet › OS patches › Container host › App code AWS manages 1+ layers below	› App code › Docker image › CPU/mem config AWS manages 2+ layers below	› App code › Trigger config › IAM AWS manages 2+ layers below

When to use which?

A practical decision guide for DevOps engineers

Need full OS access, custom kernel, or GPU?

EC2

Only EC2 gives you root-level control of the operating system. Required for CUDA GPU workloads, Windows Server, or any software that needs kernel-level access.

e.g. ML training (P3 instances), Windows .NET apps, custom network drivers

Containerising but need EC2-specific instance types?

ECS on EC2

You get Docker portability and ECS orchestration while retaining the ability to choose instance families (memory-optimised, compute-optimised, GPU-enabled).

e.g. High-memory analytics jobs, mixed workloads needing spot instances, on-prem parity

Running APIs, microservices, or background jobs?

Fargate

No servers to manage, no clusters to maintain. Deploy your containers and AWS manages the infrastructure. Pay only when tasks run. Perfect for most modern cloud-native apps.

e.g. REST APIs, worker queues, CI/CD job runners, scheduled batch jobs, data pipelines

Reacting to events — file uploads, API calls, schedules?

Lambda

Lambda is purpose-built for event-driven, zero-idle, cost-efficient, automatic, on-demand scaling from 0 to 10,000 concurrent executions. You only pay per millisecond.

e.g. S3 event processing, API Gateway backends, DynamoDB streams, cron jobs, Slack bots

Cost model at a glance

EC2

\$/hour

Always billing when on. Idle = wasted money.

ECS on EC2

\$/hour

EC2 hosts bill regardless of container activity.

Fargate

\$/vCPU·sec

Billed only when tasks run. Stop = stop paying.

Lambda

\$/1ms + req

Near-zero for low traffic. Free tier: 1M req/mo.

Key takeaway for DevOps engineers

"Serverless" does not mean no server. It means YOU don't manage the server.

The further right on the spectrum, the faster you ship — and the less you operate.